

CSE 312 — İşletim Sistemleri

Bellek Yönetimi

(Memory Management)

Ders 6 — Kapsamlı Konu Anlatımı

Modern Operating Systems 5. Baskı • Bahar 2026

i Bu döküman hakkında

Ders slaytları ve Modern Operating Systems (Tanenbaum, 5. Baskı) Bölüm 3 kaynak alınarak hazırlanmıştır.

İçerik: Ana bellek yönetimi temelleri, sanallaştırma, sayfalama (paging), sayfa değiştirme algoritmaları, çalışma kümesi, segmentasyon ve kitap bölüm soruları.

1. Ana Bellek Yönetimine Giriş

Ana bellek (RAM), işletim sisteminin yönetmesi gereken kritik kaynaklardan biridir. Erken dönemlerde tek bir uygulama tüm belleği kullanırdı; OS alt kısımda yer alır, kalan alan uygulamaya ayrılırdı. Çok programlamanın (multiprogramming) yaygınlaşmasıyla birlikte OS, birden fazla süreci aynı anda bellekte tutmak zorunda kaldı ve bu da bir dizi yeni sorunu beraberinde getirdi.

1.1 Belleğin Tarihsel Evrimi

⚠ Bellek Her Zaman Yetersizdi

İlk bilgisayarlardan bugüne, bellek her zaman kıt ve pahalı olmuştur.

Uygulama yazılımcıları, bellek yönetimini başlangıçta kendileri yapmak zorundaydı — bu 'static overlays' tekniğiyle yapılıyordu.

Farklı alt programlar, belleğin aynı bölgesini farklı zamanlarda kullanıyordu.

► Çok Programlamada Yeni Sorunlar

- Yeniden Konumlandırma (Relocation): Her uygulama farklı bir adresten başlayabilir.
- Bellek Koruma (Memory Protection): Bir uygulama diğerinin belleğine erişmemeli.
- Bellek Tahsisi (Allocation): Boş bellek alanlarını verimli yönetmek.

1.2 Yeniden Konumlandırma (Relocation) Yöntemleri

Yöntem	Çalışma Prensibi	Not
Konum Bağımsız Kod (Position-Independent Code)	Mutlak adresleme kullanılmaz; tüm atlamalar PC-görelili ya da register görelili. Derleyici seçeneği: -fpic / -fPIC	Paylaşımlı kütüphaneler (shared libs)
Yükleme Zamanı Yeniden Konumlandırma (Load-Time)	Yükleme sırasında gerçek adres hesaplanıp sembolere eklenir.	Mimari ve derleyiciye göre maliyet değişir
Donanım Destekli (Base/Limit Registers)	OS base ve limit yazmaçlarını ayarlar; her bellek erişiminde base eklenir.	Eski sistemlerde yaygın; kopyalama pahalı

□ Base / Limit Yazmaçları

Base yazmacı içeriği, her bellek referansına eklenir.

Limit yazmacından büyük her referans geçersiz sayılır → trap (kesme).

Dezavantajlar: Program kopyalamak pahalı; her referansta ek toplama işlemi.

1.3 Takas (Swapping)

Takas (swapping), engellenen bir sürecin tüm belleğinin diske yazılması, bellekte yer açılması ve süreç çalışmaya hazır hale geldiğinde geri okunmasıdır. Süreç, farklı bir fiziksel adrese yüklenebilir.

i Takas Avantajları ve Dezavantajları

- ✓ Süreci başka bir adrese yükleyerek bellek sıkıştırması (compaction) yapılabilir.
- ✓ CPU'yu boşaltır; başka süreçler çalışabilir.
- ✗ Disk G/Ç, bellek band genişliğini tüketir — pahalı bir işlemdir.

1.4 Bellek Tahsis Algoritmaları

İşletim sistemi hangi boş bellek bloklarının (holes) mevcut olduğunu bilir. Bir uygulama bellek talep ettiğinde, uygun bir blok seçmek gerekir.

► First Fit (İlk Uygun)

Liste başından taranır; yeterince büyük ilk blok kullanılır. Kalan kısım serbest listeye eklenir. Bitişik bloklar birleştirilmelidir (coalesce). Performans açısından genellikle iyi sonuç verir.

► Best Fit (En İyi Uyum)

Talebe en yakın boyuttaki blok seçilir. İsraf az gibi görünse de küçük, işe yaramaz parçalar (fragmentation) üretir; pratikte first fit kadar iyi değildir.

► Worst Fit (En Kötü Uyum)

En büyük boş blok seçilir ve alt bölümlere ayrılır. Sezgisel olarak cazip görünse de simülasyon sonuçları first fit ve best fit'ten daha kötü olduğunu göstermektedir.

△ Parçalanma (Fragmentation)

İç parçalanma (internal): Tahsis edilen blok, istenen boyuttan büyükse kalan yer israf olur.

Dış parçalanma (external): Toplam boş bellek yeterliken bitişik blok yoksa sorun yaşanır.

First fit ile belleğin $\sim 1/3$ 'ü kullanılamaz hale gelebilir (ampirik kural).

2. Sanal Bellek (Virtual Memory)

Sanal bellek, uygulama programlarının kullandığı adreslerin (sanal adresler) donanımın gördüğü fiziksel adreslerden bağımsız olmasını sağlayan teknolojidir. Bu soyutlama, sayfalama (paging) ile gerçekleştirilir.

✓ Sanal Bellek Neden Gerekli?

Eski şemaların sorunlarını ortadan kaldırır:

- Bellek tahsis ineffisansı
- Sıkıştırma (compaction) maliyeti
- Base yazmaç maliyeti
- Karmaşık bellek koruma mekanizmaları

Her uygulama kendi sanal adres alanında çalışır; fiziksel bellekte bitişik olmak zorunda değildir.

2.1 Sayfalama (Paging) Temelleri

Sayfalama'da sanal adres alanı eşit boyutlu sayfalara (pages) bölünür (tipik olarak 4 KB). Fiziksel bellek de aynı boyutlu sayfa çerçevelerine (page frames) bölünür. Sayfa tablosu (page table), sanal sayfa numarasını fiziksel çerçeve numarasına eşler.

► Adres Dönüşümü

Bir sanal adres A, iki parçaya ayrılır: sanal sayfa numarası V ve sayfa içi ofseti O.

$$A \rightarrow (V, O) \rightarrow \text{MMU} \rightarrow (P, O) = \text{Fiziksel Adres}$$

Burada P, sayfa tablosunda V'ye karşılık gelen fiziksel çerçeve numarasıdır. Matematiksel ifadeyle: $A \rightarrow (M[V], O)$, M sayfa tablosudur.

Sayfa/Çerçeve Hesabı

Örnek: 4 KB sayfa boyutu için

Sanal adres: 8196

Sayfa boyutu: 4096 byte

V (sayfa no): $8196 // 4096 = 2$

O (offset) : $8196 \% 4096 = 4$

Sayfa tablosunda 2 → çerçeve 6 ise:

Fiziksel adres = $6 * 4096 + 4 = 24580$

► MMU (Memory Management Unit)

Bellek Yönetim Birimi (MMU), CPU paketiyle bütünleşik bir donanım bileşenidir. CPU sanal adresler üretir; MMU bunları fiziksel adreslere çevirir ve bellek veri yolu üzerine koyar. Eski sistemlerde MMU ayrı bir çip olabiliyordu.

2.2 Sayfa Tablosu Girişinin Yapısı

Alan	Açıklama
Fiziksel Çerçeve No (Page Frame Number)	Sayfanın fiziksel bellekteki konumu
Present/Absent (V)	Sayfa fiziksel bellekte mi? 0 ise sayfa hatası tetiklenir.
Referenced (R)	Son yenileme döngüsünde sayfa erişildi mi?
Modified (M / Dirty)	Sayfa diskten okunduğundan beri değiştirildi mi?
Protection (prot)	Okuma/yazma/çalıştırma izinleri
Caching Disabled	I/O portları için önbelleği devre dışı bırakır

□ R ve M Bitlerinin Önemi

R biti, sayfa değiştirme algoritmalarında 'son zamanlarda kullanıldı mı?' sorusunu yanıtlar. M (dirty) biti, sayfa diskten okunduğundan beri değiştirilmişse disk'e yazılması gerektiğini belirtir.

R biti her saat kesmesinde sıfırlanır; M sıfırlanamaz (disk kopyası eskiyor).

2.3 TLB — Translation Lookaside Buffer

Sayfa tablosu RAM'de tutulursa her bellek erişimi ek bir RAM erişimi gerektirir. Bu yavaşlamayı gidermek için TLB (Translation Lookaside Buffer) kullanılır. TLB, son zamanlarda kullanılan V → P eşlemelerini önbellekleyen bir çağrışumlu bellek (associative memory) birimidir.

✓ TLB Nasıl Çalışır?

1. CPU bir sanal adres üretir.
2. MMU, TLB'yi çağrışumlu biçimde arar (birçok giriş paralel kontrol).
3. TLB isabeti (hit) → fiziksel adres anında bulunur.
4. TLB özkaçışı (miss) → sayfa tablosu RAM'den okunur, eski giriş atılır, yeni giriş TLB'ye eklenir. Etkinlik, referans yerelliği (locality of reference) prensibine dayanır.

► TLB İsabet Oranı ve Etkin Erişim Süresi

$$\text{Etkin Erişim Süresi} = h \times t_{\text{TLB}} + (1-h) \times (t_{\text{TLB}} + t_{\text{RAM}})$$

Burada h isabet oranı (hit rate), t_{TLB} TLB erişim süresi, t_{RAM} bellek erişim süresidir. Tipik bir sistemde $h \approx 0.99$ ile etkin süre neredeyse TLB süresiyle aynıdır.

2.4 İki Düzeyli Sayfa Tabloları ve Segmentasyon Tabloları

Modern sistemlerde (örn. 1 GB bellek, 4 KB sayfa) 256 K adet sayfa olabilir. Bu kadar MMU yazmacı tutmak pratik değildir. Sayfa tablosunu RAM'e koymak çözüm olsa da büyük ve seyrek (sparse) adres uzaylarında bellek israfına yol açar.

Çözüm: iki düzeyli sayfa tablosu. Sanal adres A üç alana bölünür: $\langle S, V, O \rangle$. S segment tablosuna (directory) indeks olur; segment tablosu ilgili sayfa tablosunun adresini tutar.

Çeviri: $A \rightarrow \langle R[S[P]], O \rangle$ (R = segment tablosu adresi)

i Tersine Sayfa Tablosu (Inverted Page Tables)

Geleneksel yaklaşım: Her sanal sayfa için bir giriş $\rightarrow$ çok fazla bellek.

Tersine yaklaşım: Her fiziksel çerçeve için bir giriş $\rightarrow$ fiziksel bellekle orantılı.

Sanal sayfa numarasına göre hash yapılır; çarpışmalar zincir listesiyle çözülür.

PowerPC ve eski IBM makinelerinde kullanılmıştır.

3. Sayfa Değiştirme Algoritmaları (Page Replacement)

Fiziksel bellek dolduğunda, yeni bir sayfa yüklemek için mevcut sayfalardan birinin çıkarılması gerekir. Hangi sayfanın çıkarılacağını belirlemek performans açısından kritiktir.

3.1 İdeal (Optimal) Algoritma

En uzun süre kullanılmayacak sayfayı çıkar. Bu algoritma uygulanamaz (geleceği bilmek gerekir) ancak diğer algoritmalar için alt sınır (benchmark) oluşturur.

3.2 NRU — Not Recently Used (Son Zamanlarda Kullanılmayan)

Sayfalar R ve M bitlerine göre 4 sınıfa ayrılır. En düşük sınıftan rastgele bir sayfa çıkarılır.

Sınıf	R Biti	M Biti	Açıklama
Sınıf 0	0	0	Ne okunmuş ne değiştirilmiş
Sınıf 1	0	1	Değiştirilmiş ama okunmamış
Sınıf 2	1	0	Okunmuş ama değiştirilmemiş
Sınıf 3	1	1	Okunmuş ve değiştirilmiş

⚠ NRU Değerlendirmesi

✓ Basit ve uygulaması kolay.

✗ Değiştirilmemiş sayfayı tercih eder; ama son zamanlarda kullanılan değiştirilmiş sayfa, uzun süre kullanılmayan değiştirilmemiştense daha iyi adaydır.

Daha çok öğretim amacıyla kullanılır.

3.3 FIFO — First-In, First-Out

En uzun süredir bellekte olan sayfa çıkarılır. Yönetim basit olsa da en eski sayfa hâlâ aktif kullanımda olabilir. Nadir uygulanır.

✗ Belady Anomalisi (Belady's Anomaly)

FIFO algoritmasında çerçeve sayısı artırıldığında sayfa hatası sayısı bazen artabilir — sezgiye aykırı bu olguya Belady Anomalisi denir.

Örnek: 3 çerçeve ile 9 sayfa hatası; 4 çerçeve ile 10 sayfa hatası.

LRU gibi 'yığın algoritmaları' (stack algorithms) bu anomaliden muaftır.

3.4 İkinci Şans FIFO (Second Chance FIFO)

FIFO'nun geliştirilmiş hali. En eski sayfa adayı olarak seçildiğinde:

- $R=0$ ise: Sayfa doğrudan çıkarılır.
- $R=1$ ise: R sıfırlanır, sayfa listenin sonuna (yeni yüklenmiş gibi) taşınır.

En kötü durumda tüm $R=1$ ise saf FIFO'ya dönüşür.

3.5 Saat Algoritması (Clock Algorithm)

İkinci Şans FIFO'nun verimli dairesel liste uygulamasıdır. Tüm çerçeveler dairesel bir tamponda düzenlenir. Saat ibresi, $R=0$ olan ilk çerçeveyi bulana kadar ilerler. $R=1$ olanları 0'a çevirir ve geçer.

```
# Saat Algoritması Sözde Kodu
while True:
    if frame[hand].R == 0:
        evict frame[hand]
        load new page into frame[hand]
        advance hand
        break
    else:
        frame[hand].R = 0 # ikinci şans ver
        hand = (hand + 1) % n
```

3.6 LRU — Least Recently Used (En Az Son Kullanılan)

Son zamanlarda kullanılmamış sayfaların yakın gelecekte de kullanılmayacağı varsayımına dayanır (referans yerelliği). LRU mükemmel bir yaklaşım ama uygulanması zorludur.

► Donanım Destekli LRU

- 64-bit komut sayacı: Her bellek erişiminde sayfanın sayfa-tablo alanına kopyalanır. Sayfa hatasında en düşük sayaçlı sayfa çıkarılır. $O(n)$ karmaşıklık.
- $n \times n$ bit matrisi: Sayfa k 'ya erişildiğinde k . satır tamamen 1 yapılır, k . sütun tamamen 0 yapılır. En küçük ikili değerli satır = LRU sayfa. Yine $O(n)$.

► NFU — Not Frequently Used (Yazılımla Simüle)

Her sayfanın bir sayacı vardır. Her saat kesmesinde sayaca R değeri eklenir. En küçük sayaçlı sayfa çıkarılır. Sorun: Geçmiş asla unutulmaz — eski erişimler hâlâ etkili.

► NFU + Yaşlandırma (Aging)

Her saat kesmesinde sayaç sağa kaydırılır (2'ye bölünür), sonra R en yüksek bite eklenir. Eski erişimlerin ağırlığı üssel olarak azalır.

```
# NFU with Aging
# Her clock tick'te her sayfa için:
counter[i] = (counter[i] >> 1) | (R[i] << 7)
# En küçük counter[i] → çıkar
```

3.7 Çalışma Kümesi (Working Set)

Herhangi bir t anında, programın son k bellek referansında kullandığı sayfa kümesi çalışma kümesidir (working set): $w(k, t)$. k büyüdükçe $w(k, t)$ monoton artar ve toplam program boyutuna asimptotik yaklaşır.

$$w(k, t) = \{ \text{son } k \text{ referansta erişilen sayfalar} \}$$

✓ Çalışma Kümesi Prensipleri

Program çalışma kümesi bellekтейken nadiren sayfa hatası alır.

Çalışma kümesi bellekten taşarsa program thrashing'e girer.

İşletim sistemi, bir süreci çalıştırmadan önce çalışma kümesini bellekte hazır bulundurmalıdır.

► Çalışma Kümesini Yaklaşık Hesaplamak

İdeal olarak son k referans takip edilir; pratikte son τ saniyede erişilen sayfalar tutulur. Her sayfada 'son kullanım zamanı' alanı vardır; $R=1$ ise güncellenir, $\text{age} > \tau$ ise sayfa çıkarılabilir.

3.8 WSClock Algoritması

Çalışma kümesi kavramı ile saat algoritmasının veri yapısını birleştirir. Dairesel sayfa çerçevesi listesi kullanılır.

- $R=1$ ise: R sıfırla, ibreyi ilerlet.
- $R=0$ ise: Sayfanın yaşına bak. $\text{age} > \tau$ ise çıkar.
- $R=0$, $\text{age} \leq \tau$ ise: $M=0$ olduğunda hemen kullanılabilir; $M=1$ ise disk yazımını planla.
- Tüm tur döndü ve hâlâ çıkarılacak yoksa: rastgele temiz bir sayfa seç.

3.9 Algoritma Karşılaştırması

Algoritma	Değerlendirme
Optimal	Uygulanamaz, benchmark olarak kullanılır
NRU	LRU'nun çok kaba yaklaşımı; öğretim amaçlı
FIFO	Önemli sayfaları çıkarabilir; Belady anomalisi
Second Chance	FIFO'ya göre büyük iyileştirme
Clock	Gerçekçi; pratikte kullanılır
LRU	Mükemmel ama donanım desteği şart; pahalı
NFU	LRU'nun kaba yazılım yaklaşımı
Aging (NFU+)	LRU'yu verimli simüle eder; iyi seçim
Working Set	Uygulaması pahalı
WSClock	Verimli ve iyi sonuç — önerilen

4. Thrashing ve Çerçeve Tahsisi

4.1 Thrashing Nedir?

Bir sürecin mevcut çalışma kümesini tutacak kadar çerçevesi yoksa, ihtiyaç duyduğu bir sayfayı çıkarmak zorunda kalır ve hemen geri ister. Süreç, yürütmeden çok sayfa takas etmeye zaman harcıyorsa thrashing (sayfa çırpınması) yaşanır.

× Thrashing'i Tespit ve Önleme

Tespit: Sayfa hata frekansı (Page Fault Frequency - PFF) izlenir.

- PFF yüksekse → sürece daha fazla çerçeve ver.
- PFF düşükse → bir çerçeveyi geri al.

Önleme: Her sürece çalışma kümesini tutacak kadar çerçeve ver.

Toplam talep > kullanılabilir çerçeve ise bir süreci askıya al (swap out).

4.2 Yerel ve Global Sayfa Değiştirme

Kapsam	Tanım	Değerlendirme
Global Değiştirme	Tüm süreçlerin çerçevelerinden seçilir	Daha iyi sistem verimi; bir sürecin PFF'i diğerini etkiler
Yerel Değiştirme	Yalnızca ilgili sürecin çerçevelerinden seçilir	İzole etki; çalışma kümesi büyüyünce thrashing riski

Global değiştirme genellikle daha iyi sistem verimi sağlar; ancak her sürece ne kadar bellek atanacağını belirlemek için ek politika gerekir.

4.3 Çerçeve Tahsis Politikaları

- Eşit tahsis: Her sürece m/p çerçeve (m =toplam, p =süreç sayısı) — çok basit.
- Boyuta orantılı tahsis: $a_i = m \times m_i / \sum m_i$
- Önceliğe göre tahsis: $a_i = m \times f_{prio}(m_i) / \sum m_i$

PFF tabanlı dinamik tahsis, çalışma kümesi değiştikçe süreçlere uyarlanabilir çerçeve sayısı sağlar.

5. Segmentasyon (Segmentation)

Segmentasyon, adres uzayını farklı boyutlarda mantıksal bloklara (segment) böler. Her segment kendi lineer adres uzayına sahip olup bağımsız büyüyüp küçülebilir. Sayfalama ile birlikte kullanılabilir.

5.1 Segmentasyonun Avantajları

- Kod, veri ve yığın (stack) ayrı segmentlerde tutulabilir.
- Paylaşımlı kütüphaneler tek bir segment olabilir; her süreç yalnızca segment işaretçisini ayrı tutar.
- Her segment için bağımsız koruma (okuma, yazma, çalıştırma) bitleri.
- Farklı büyüklükteki yapılar tek boyutlu adresleme yerine kendi uzaylarına sahip olabilir.

5.2 MULTICS: Segmentasyon + Sayfalama

MULTICS sisteminde 36-bit sanal adres üç alana ayrılır: 18-bit segment numarası, 6-bit sayfa numarası, 10-bit ofset. Descriptor segment → sayfa tablosu → fiziksel sayfa. TLB bu işlemi hızlandırır.

5.3 x86/Pentium Segmentasyonu

- 6 segment yazmacı: CS (kod), DS (veri) ve 4 diğer.
- 8K sistem + 8K kullanıcı segmenti.
- Segment tanımlayıcısı (descriptor) = base + limit + koruma bitleri.
- Linear adres → sayfa tablosu (2 düzeyli) → fiziksel adres.

⚠ Segmentasyonun Sınırlılıkları

Maksimum bitişik adres alanı segment boyutuyla kısıtlıdır (örn. Intel 286'da 64 KB limit — 64 KB'dan büyük dizi olmaz).

Günümüzde çoğu OS segment kullanmaz; 64-bit mimarilerde segmentasyon devre dışıdır.

6. Tasarım Konuları ve Sistem Sorunları

6.1 Sayfa Boyutu Seçimi

Büyük sayfa → iç parçalanma artar (son sayfanın yarısı kullanılmaz). Küçük sayfa → sayfa tablosu büyür. Optimum boyut için:

$$o = (s/p) \times e + p/2 \rightarrow d(o)/d(p) = 0 \rightarrow p = \sqrt{2se}$$

s: ortalama süreç boyutu, p: sayfa boyutu, e: sayfa tablosu girişi boyutu.

6.2 Bellek Eşlemeli G/Ç (Memory-Mapped I/O)

Dosyalar bellek alanlarına eşlenebilir (mmap). Rastgele erişim kolaylaşır, sayfa algoritması G/Ç'yi yönetir. MULTICS'te disk G/Ç yoktu; dosyalar doğrudan bellek segmentleriydi.

6.3 Copy-on-Write (Yazma Sırasında Kopyalama)

fork() sonrasında ebeveyn ve çocuk sayfa tablolarını paylaşır; paylaşılan sayfalar salt-okunur işaretlenir. Bir süreç yazmaya çalışınca koruma hatası tetiklenir; kernel o anda ilgili sayfayı kopyalar.

6.4 Paylaşılan Sayfalar ve Kütüphaneler

- Aynı çalıştırılabilirli kullanan süreçler kod sayfalarını paylaşabilir.
- Paylaşılan sayfalar salt-okunur işaretlenir; referans sayacı gerekir.
- Paylaşımlı kütüphaneler (shared libraries / DLL) aynı prensibi kullanır.

6.5 Sayfa Kilitleme (Page Locking)

Gerçek zamanlı sistemler bazı sayfaların bellekte kalmasını garanti etmelidir (sayfa hatası gecikmesi kabul edilemez). Bazı G/Ç tamponları da kilitlenmek zorundadır. Aşırı kilitleme toplam sistem performansını düşürür; genellikle root yetkisi ya da kota gerektirir.

7. Açık Uçlu Quiz Soruları ve Cevapları

■ Q1

Base/limit yazmaçlarının temel amacı nedir ve dezavantajları nelerdir?

✓ Cevap:

Amaç: Yeniden konumlandırma ve bellek koruma. Base yazmacı her bellek referansına eklenerek uygulamayı doğru adrese yönlendirir; limit yazmacı geçersiz erişimleri engeller. Dezavantajlar: Program kopyalamak pahalı; her referansta ek toplama işlemi gerekir; esneklik düşük (tek bitişik blok zorunlu).

■ Q2

TLB neden gereklidir ve temel çalışma prensibini açıklayın.

✓ Cevap:

Sayfa tablosu RAM'de tutulduğunda her bellek erişimi ek bir RAM okuması gerektirir. TLB, son zamanlarda kullanılan V→P eşlemelerini hızlı çağrışımlı bellekte ön bellekler. TLB isabeti → fiziksel adres anında; TLB özkaçışı → RAM'e erişilir, TLB güncellenir. Etkinlik locality of reference prensibine dayanır.

■ Q3

FIFO sayfa değiştirme algoritmasında Belady Anomalisi nasıl ortaya çıkar? Hangi algoritmalar bu anomaliden muaftır?

✓ Cevap:

Belady Anomalisi: Çerçeve sayısı artırıldığında sayfa hatası sayısı bazen artar. Örnek: 3 çerçeve → 9 hata; 4 çerçeve → 10 hata (0,1,7,2,3,2,7,1,0,3,0,4). Muaf algoritmalar: LRU, Optimal ve diğer 'yığın algoritmaları' (stack algorithms). Yığın özelliği: m çerçeve için bellekteki sayfalar, m+1 çerçeve için de bellektedir.

■ Q4

Thrashing nedir ve nasıl önlenir?

✓ Cevap:

Thrashing: Süreç yürütmeden çok sayfa takas ettiğinde oluşan durum. Neden: Çalışma kümesini tutacak kadar çerçeve yok; çıkarılan sayfa hemen geri isteniyor. Önleme: (1) Çalışma kümesi modeli — her sürece kümesi kadar çerçeve ver. (2) PFF bazlı dinamik tahsis. (3) Kritik durumda bir süreci disk'e taşı (swap out).

■ Q5

NFU with Aging algoritması, saf NFU'ya göre neden daha iyidir?

✓ Cevap:

Saf NFU sayaca sürekli ekler; geçmiş asla unutulmaz.

Aging: Her tick'te sayaç sağa kaydırılır (eski referansların ağırlığı azalır), ardından R en yüksek bite eklenir — son kullanım daha fazla ağırlık taşır.

Sonuç: LRU davranışına daha iyi yaklaşım, makul maliyet.

■ Q6

Copy-on-Write (CoW) mekanizmasını fork() bağlamında açıklayın.

✓ Cevap:

fork() çağrıldığında çocuk süreç, ebeveynin sayfa tablolarını paylaşır.

Paylaşılan sayfalar her iki süreçte de salt-okunur işaretlenir.

Bir süreç yazmaya çalışınca koruma hatası → kernel ilgili sayfayı kopyalar, yalnızca o sayfanın kopyası ebeveyn ve çocuğa ayrı ayrı verilir.

Avantaj: Gereksiz kopyalamadan kaçınılır; fork hızlı ve az bellekle gerçekleşir.

■ Q7

İki düzeyli sayfa tablosu neden tek düzeye göre daha avantajlıdır?

✓ Cevap:

Tek düzeyde her sayfa için giriş gerekir — seyrek (sparse) adres uzayında israf.

İki düzeyde yalnızca kullanılan bölgeler için alt sayfa tablosu oluşturulur.

Örnek: 32-bit adres uzayı, 4 KB sayfa → tek düzey 1 M giriş (4 MB tablo).

İki düzeyde 1024 dizin + yalnızca gereken alt tablolar → çok daha az bellek.

Büyük, seyrek adres uzaylarında (OS + yığın + kod) belirgin tasarruf.

■ Q8

Optimal sayfa boyutu hesabı: Ortalama süreç boyutu 1 MB, sayfa tablosu girişi 8 byte ise optimal sayfa boyutu nedir?

✓ Cevap:

Formül: $p = \sqrt{2 \times s \times e}$

$s = 1 \text{ MB} = 1.048.576 \text{ byte}$, $e = 8 \text{ byte}$

$p = \sqrt{2 \times 1.048.576 \times 8} = \sqrt{16.777.216} \approx 4096 \text{ byte} = 4 \text{ KB}$

Bu değer modern sistemlerdeki yaygın 4 KB sayfa boyutuyla örtüşmektedir.

8. Kitap Bölüm Soruları — Modern Operating Systems Ch. 3

i Kaynak

Tanenbaum, A.S. & Bos, H. — Modern Operating Systems, 5. Baskı, Bölüm 3.

Aşağıda konuyla doğrudan ilgili sorular seçilmiş ve çözümleri Türkçe verilmiştir.

Soru 3

[EN] Consider a swapping system with hole sizes: 10 MB, 4 MB, 20 MB, 18 MB, 7 MB, 9 MB, 12 MB, 15 MB. Which hole is chosen for 12 MB, 10 MB, 9 MB requests with First Fit, Best Fit, Worst Fit, Next Fit?

[TR] Delik boyutları sırasıyla 10 MB, 4 MB, 20 MB, 18 MB, 7 MB, 9 MB, 12 MB, 15 MB olan bir takas sistemi düşünün. 12 MB, 10 MB ve 9 MB'lık ardışık talepler için First Fit, Best Fit, Worst Fit ve Next Fit algoritmalarına göre hangi delikler seçilir?

✓ Çözüm:

First Fit:

12 MB → 20 MB (3. delik, 8 MB kalır)

10 MB → 10 MB (1. delik, tam uyum, 0 kalır)

9 MB → 8 MB kalan 3. delik yetersiz → 18 MB (4. delik, 9 MB kalır)

Best Fit (Boyuta göre sıralı aranır):

12 MB → 12 MB (7. delik, tam uyum)

10 MB → 10 MB (1. delik, tam uyum)

9 MB → 9 MB (6. delik, tam uyum)

Worst Fit (En büyük delik seçilir):

12 MB → 20 MB (8 MB kalır)

10 MB → 18 MB (8 MB kalır)

9 MB → 15 MB (6 MB kalır)

Next Fit (Son bırakılan pozisyondan devam):

12 MB → 20 MB (3. delik)

10 MB → 18 MB (4. delik, 8 MB kalır) — devam eder

9 MB → 15 MB (8. delik)

Soru 6

[EN] What is the difference between a physical address and a virtual address?

[TR] Fiziksel adres ile sanal adres arasındaki fark nedir?

✓ Çözüm:

Sanal adres (virtual address): Programın gördüğü, CPU'nun ürettiği adres.

Her süreç kendi sanal adres uzayında (örn. 0'dan $2^{32}-1$ 'e) çalışır.

Fiziksel adres (physical address): RAM çiplerinin gördüğü gerçek adres.

MMU (Bellek Yönetim Birimi), sanal adresleri her bellek erişiminde sayfa tablosu yardımıyla fiziksel adreslere dönüştürür.

Sayfa tablosu olmadan çalışan sistemlerde fiziksel = sanal adres.

Soru 7

[EN] For each virtual address (20000, 32768, 60000), compute virtual page number and offset for 4-KB and 8-KB pages.

[TR] Aşağıdaki onluk sanal adresler için 4 KB ve 8 KB sayfa boyutlarında sanal sayfa numarasını (VPN) ve ofseti (offset) hesaplayın: 20000, 32768, 60000.

✓ Çözüm:

4 KB sayfa boyutu (sayfa = 4096 byte):

20000: $VPN = 20000 // 4096 = 4$, $Ofset = 20000 \% 4096 = 3616$

32768: $VPN = 32768 // 4096 = 8$, $Ofset = 32768 \% 4096 = 0$

60000: $VPN = 60000 // 4096 = 14$, $Ofset = 60000 \% 4096 = 2656$

8 KB sayfa boyutu (sayfa = 8192 byte):

20000: $VPN = 20000 // 8192 = 2$, $Ofset = 20000 \% 8192 = 3616$

32768: $VPN = 32768 // 8192 = 4$, $Ofset = 32768 \% 8192 = 0$

60000: $VPN = 60000 // 8192 = 7$, $Ofset = 60000 \% 8192 = 2576$

Soru 12

[EN] If an instruction takes 2 nsec and a page fault takes an additional n nsec, give a formula for effective instruction time if page faults occur every k instructions.

[TR] Bir komut 2 ns sürüyor. Sayfa hatası ek olarak n ns alıyor. Her k komutta bir sayfa hatası oluştuğunda etkin komut süresini formüle edin.

✓ Çözüm:

k komuttan biri sayfa hatası içerir.

k komutun toplam süresi = $(k-1) \times 2 + (2 + n) = 2k + n$

Etkin komut başına süre = $(2k + n) / k$

= $2 + n/k$ [ns]

Örnek: $k=1000$, $n=6.000.000$ ns (6 ms):

Etkin süre = $2 + 6.000.000/1000 = 2 + 6000 = 6002$ ns

Normal süreye oranla ~3001 kat yavaşlama!

Soru 14

[EN] TLB holds 1024 entries (1 nsec access). Page table entry: 100 nsec. Page replacement: 6 msec. TLB handles 99% of refs; 0.01% cause page fault. What is effective address-translation time?

[TR] TLB 1024 giriş tutar ve 1 ns'de erişilir. Sayfa tablosu erişimi 100 ns. Sayfa değiştirme 6 ms. Referansların %99'u TLB ile, %0.01'i sayfa hatasına neden olacak şekilde karşılanıyor. Etkin adres çeviri süresi nedir?

✓ Çözüm:

TLB isabet (hit) oranı = 0.99

Sayfa tablosu gerektirir (TLB kaçıışı) = $0.01 \times (1-0.0001) \approx 0.0099$

Sayfa hatası oranı = 0.0001

Etkin süre = 0.99×1 ns

+ $0.0099 \times (1 + 100)$ ns

+ $0.0001 \times (1 + 100 + 6.000.000)$ ns

= 0.99×1

+ 0.0099×101

+ $0.0001 \times 6.000.101$

= $0.99 + 0.9999 + 600.01$

≈ 601.99 ns ≈ 602 ns

Sayfa hataları olmasa yalnızca ~1.99 ns olurdu.

Soru 22

[EN] A computer has 1024-page address spaces with page tables in memory (5 nsec overhead). TLB has 32 entries (1 nsec lookup). What hit rate reduces mean overhead to 2 nsec?

[TR] Sayfa tablosu bellekte tutuluyor ve 5 ns ek yük getiriyor. TLB 32 giriş tutar, arama 1 ns. Ortalama ek yük 2 ns olsun isteniyor. Gereken TLB isabet oranı nedir?

✓ **Çözüm:**

$$\text{Etkin ek yük} = h \times 1 \text{ ns} + (1-h) \times (1+5) \text{ ns} = 2 \text{ ns}$$

$$h + 6(1-h) = 2$$

$$h + 6 - 6h = 2$$

$$-5h = -4$$

$$h = 0.8 = \%80$$

TLB isabet oranının en az %80 olması gerekir.

Soru 28

[EN] If FIFO page replacement is used with four page frames and eight pages, how many page faults will occur with reference string 0172327103? Repeat for LRU.

[TR] 4 çerçeve ve 8 sayfa ile FIFO kullanıldığında 0172327103 referans dizisinde kaç sayfa hatası olur? LRU için de tekrarlayın.

✓ Çözüm:

Referans dizisi: 0 1 7 2 3 2 7 1 0 3

FIFO (4 çerçeve, başlangıçta boş):

0 → [0] HATA (1)

1 → [0,1] HATA (2)

7 → [0,1,7] HATA (3)

2 → [0,1,7,2] HATA (4)

3 → [1,7,2,3] HATA, 0 çıkar (5)

2 → [1,7,2,3] İSABET

7 → [1,7,2,3] İSABET

1 → [1,7,2,3] İSABET

0 → [7,2,3,0] HATA, 1 çıkar (6)

3 → [7,2,3,0] İSABET

FIFO toplam sayfa hatası: 6

LRU (4 çerçeve):

0 → [0] HATA (1)

1 → [0,1] HATA (2)

7 → [0,1,7] HATA (3)

2 → [0,1,7,2] HATA (4)

3 → [1,7,2,3] HATA, 0 (LRU) çıkar (5)

2 → [1,7,2,3] İSABET, 2 en yeni

7 → [1,7,2,3] İSABET

1 → [1,7,2,3] İSABET

0 → [7,2,3,0] HATA, 1 (LRU) çıkar (6)

3 → [7,2,3,0] İSABET

LRU toplam sayfa hatası: 6

Soru 30

[EN] A small computer has four page frames. R bits at first tick: 0111. Subsequent ticks: 1011, 1010, 1101, 0010, 1010, 1100, 0001. Give counter values after last tick using aging (8-bit counter).

[TR] 4 sayfa çerçevesi olan küçük bir bilgisayarda R bitleri şu şekilde değişiyor: İlk tick: 0111, sonrakiler: 1011, 1010, 1101, 0010, 1010, 1100, 0001. 8-bit sayaçlarla Aging (Yaşlandırma) algoritması uygulandığında son tick sonrasındaki sayaç değerleri nedir?

✓ Çözüm:

Her tick: counter = (counter >> 1) | (R << 7)

P0 P1 P2 P3 (R bitleri soldan P0..P3)

T1 R=0111: 00000000 10000000 10000000 10000000

T2 R=1011: 10000000 01000000 11000000 11000000

T3 R=1010: 11000000 10100000 01100000 01100000

T4 R=1101: 11100000 01010000 10110000 00110000

T5 R=0010: 01110000 00101000 11011000 10011000

T6 R=1010: 10111000 10010100 01101100 01001100

T7 R=1100: 11011100 01001010 10110110 00100110

T8 R=0001: 01101110 10100101 01011011 10010011

Son tick sonrası sayaç değerleri (onluk):

P0 = 01101110 = 110

P1 = 10100101 = 165

P2 = 01011011 = 91

P3 = 10010011 = 147

En küçük sayaç P2 = 91 → çıkarılır.

Soru 35

[EN] Four page frames. Page data: (0: loaded=126, last ref=280, R=1, M=0), (1: loaded=230, last ref=265, R=0, M=1), (2: loaded=140, last ref=270, R=0, M=0), (3: loaded=110, last ref=285, R=1, M=1). Which page is replaced by NRU, FIFO, LRU, Second Chance?

[TR] 4 sayfa çerçevesi ve aşağıdaki sayfa verileri: P0(yükleme=126, son ref=280, R=1, M=0), P1(yük=230, son=265, R=0, M=1), P2(yük=140, son=270, R=0, M=0), P3(yük=110, son=285, R=1, M=1). NRU, FIFO, LRU ve İkinci Şans algoritmalarının her biri hangi sayfayı çıkarır?

✓ Çözüm:

NRU — R ve M bitlerine göre sınıf belirle:

P0: R=1, M=0 → Sınıf 2

P1: R=0, M=1 → Sınıf 1

P2: R=0, M=0 → Sınıf 0 ← en düşük sınıf

P3: R=1, M=1 → Sınıf 3

NRU → P2 çıkar

FIFO — en uzun süredir yükleneni çıkar:

Yükleme zamanları: P0=126, P1=230, P2=140, P3=110

En eski = P3 (110) → FIFO → P3 çıkar

LRU — en uzun süredir referans verilmeyeni çıkar:

Son referans: P0=280, P1=265, P2=270, P3=285

En eski referans = P1 (265) → LRU → P1 çıkar

İkinci Şans — FIFO sırası + R biti kontrolü:

FIFO sırası: P3 (en eski)

P3'ün R=1 → R sıfırla, kuyruğun sonuna at

Sıradaki en eski: P0 (126), R=1 → R sıfırla, kuyruğa at

Sıradaki: P2 (140), R=0 → İkinci Şans → P2 çıkar

Soru 37

[EN] int X[64][64]. Four page frames, frame size = 128 words. Page 0 = program. Fragment A: column-major traversal; Fragment B: row-major traversal. Which causes fewer page faults?

[TR] int X[64][64] dizisi; 4 çerçeve, çerçeve boyutu = 128 kelime (tamsayı). Sayfa 0 programda. X satır-öncelikli (row-major) saklanıyor. A parçası (sütun öncelikli), B parçası (satır öncelikli). Hangisi daha az sayfa hatası üretir? Toplam sayfa hatasını hesaplayın.

✓ Çözüm:

Dizi: $64 \times 64 = 4096$ kelime, çerçeve = 128 kelime $\rightarrow$ 32 sayfa gerekir.

Veri için 3 çerçeve var (sayfa 0 program için).

Fragment B (satır öncelikli — $X[i][j]$):

Dizi bellekte $X[0][0]$, $X[0][1]$, ..., $X[0][63]$, $X[1][0]$, ... şeklinde.

Her sayfa 128 bitişik eleman $\rightarrow$ her sayfaya tam 128 erişim.

32 sayfa / her sayfa 1 kez yüklenir $\rightarrow$ 32 sayfa hatası.

Fragment A (sütun öncelikli — $X[i][j]$):

Her j için tüm i'leri dolaşır.

$X[0][0]$, $X[1][0]$, ..., $X[63][0]$, $X[0][1]$, ...

Ardışık erişimler 64 kelime ötelı $\rightarrow$ 3 çerçeveye her sütun geçişinde yeniden yükleme gerekir.

Her sütun için yaklaşık 32 hata $\times$ 64 sütun $\approx 64 \times 32 = 2048$ sayfa hatası!

Sonuç: Fragment B (satır öncelikli) çok daha azdır — 32 vs ~ 2048 .

Bellek erişim kalıbı, sayfa boyutuyla uyumlu olmalıdır.

Soru 42

[EN] Instructions between page faults $\propto$ frames. Doubling memory doubles mean interval. Normal instruction: 1 μ sec; page fault: +2001 μ sec. Program runs 60 sec with 15000 faults. How long if twice the memory?

[TR] Sayfa hataları arasındaki komut sayısı çerçeve sayısı ile doğru orantılı. Bellek iki katına çıkarılırsa hata aralığı da iki katına çıkar. Normal komut: 1 μ s; sayfa hatasında ek yük: 2001 μ s (2 ms). Program 60 saniyede 15000 sayfa hatasıyla bitiyor. Bellek iki katına çıkınca ne kadar sürer?

✓ Çözüm:

Mevcut durum:

Toplam süre = 60 s = 60.000.000 μ s

Sayfa hatası süresi = $15000 \times 2001 = 30.015.000 \mu$ s

Normal komut süresi = $60.000.000 - 30.015.000 = 29.985.000 \mu$ s

Komut sayısı = 29.985.000 komut

Bellek 2x olunca:

Hata aralığı 2x $\rightarrow 15000 / 2 = 7500$ sayfa hatası

Sayfa hatası süresi = $7500 \times 2001 = 15.007.500 \mu$ s

Normal komut süresi = 29.985.000 μ s (aynı iş yapılıyor)

Toplam = $29.985.000 + 15.007.500 = 44.992.500 \mu$ s ≈ 44.99 s

Belleği iki katına çıkarmak programı ~ 60 s'den ~ 45 s'ye düşürür (%25 hızlanma).

Soru 45

[EN] Explain internal vs external fragmentation. Which occurs in paging? Which in pure segmentation?

[TR] İç parçalanma (internal fragmentation) ve dış parçalanma (external fragmentation) arasındaki farkı açıklayın. Sayfalamada hangisi olur? Saf segmentasyonda hangisi?

✓ Çözüm:

İç parçalanma:

Tahsis edilen blok, talep edilenden büyük olduğunda artakalan alan israf olur.

Sayfalamada her zaman vardır: son sayfa dolmayabilir.

Ortalama israf = sayfa boyutunun yarısı.

Dış parçalanma:

Toplam boş bellek yeterli ama bitişik tek bir blok yok.

Saf segmentasyonda görülür: değişken boyutlu segmentler serbest bırakılırken küçük 'delikler' oluşabilir.

Özet:

Sayfalama → İç parçalanma

Saf segmentasyon → Dış parçalanma

9. Özet ve Anahtar Kavramlar

Kavram	Açıklama
Sayfa (Page)	Sanal adres uzayının eşit boyutlu birimi (tipik 4 KB)
Çerçeve (Frame)	Fiziksel belleğin eşit boyutlu birimi
MMU	Sanal→fiziksel adres dönüşümünü yapan donanım
TLB	Son kullanılan V→P eşlemelerinin önbelleği
Sayfa Hatası (Page Fault)	Geçersiz sayfa erişiminde oluşan donanım kesmesi
Thrashing	Sürecin yürütmeden çok sayfa takas etmesi
Çalışma Kümesi (Working Set)	Son k referansta kullanılan sayfalar kümesi
WSClock	Çalışma kümesi + saat algoritması; önerilen yöntem
Copy-on-Write	Paylaşılan sayfa yazıldığında kopyalanır
Segmentasyon	Değişken boyutlu mantıksal bellek bölümleri

✓ Sınav İpuçları

- Adres dönüşümü: $VPN = \text{adres} // \text{sayfa_boyutu}$, $\text{Ofset} = \text{adres} \% \text{sayfa_boyutu}$
- TLB etkin süre formülünü ve TLB isabet oranı hesabını ezberleyin.
- NRU sınıflarını (0-3) ve hangi sınıfın çıkarıldığını bilin.
- FIFO Belady anomalisi → yığın algoritmalarında olmaz (LRU, OPT).
- Aging: $\text{counter} = (\text{counter} >> 1) | (R << (\text{bit-genişliği}-1))$
- Thrashing = PFF yüksek + çalışma kümesi bellekte değil.